HeySigApp coding specification

October 16, 2026 · Implementation draft v1 · Phone and Pixel Watch 3

1. Scope and implementation decisions

Build a native Android application and Wear OS companion to replace HeySig3. Every function below requires phone voice and watch voice access. Touch screens are an additional interface. The phone executes all hearing-aid commands through its speaker. The watch sends intents to the phone and receives status; it does not emit aid-control tones.

The program specification defines product requirements and Tasker parity. This document defines the proposed code contracts, screens and execution rules. These are design decisions to implement; the current `ProtoActivity` is a working transport experiment, not this application.

Use Kotlin, Compose, coroutines and StateFlow. Keep pure models, parsing and automation decisions in `shared`. Android adapters provide audio, microphone, speech, storage, time and Wear communication. Use dependency injection through constructors. ViewModels call use cases; they must not call the PCM generator or AudioTrack directly.

The acoustic transport has no aid readback or receipt acknowledgment. UI text and replies say “last sent,” “user confirmed,” or “unknown.” Signia is not a live state source. Explicitly separate successful PCM playback from actual aid reception.

1.1 Proposed source layout

Package / file	Responsibility
shared/model/Fitting.kt	Slot IDs, capabilities, waveform configuration and fitting revision
shared/model/Command.kt	Typed control requests, origin and request IDs
shared/model/RequestedState.kt	Program-context values and provenance
shared/model/OperationResult.kt	Typed stages, outcomes and recovery reasons
shared/voice/CommandParser.kt	Pure text-to-intent parsing and ambiguity handling
shared/voice/ConversationReducer.kt	Setup/settings conversations and pending confirmations
shared/control/CommandCoordinator.kt	Single-owner execution, cancellation and priority
shared/control/StateReducer.kt	Apply completed/uncertain operation events
shared/settings/SnapshotService.kt	Save, preview and restore plans
shared/noise/NoiseReducer.kt	Pure hysteresis, confirmation and cooldown logic
shared/sleep/SleepPlanner.kt	Next events, occurrence IDs and clock-change decisions
androidApp/audio/AcousticTransport.kt	ProtoRequest adapter and streaming playback
androidApp/audio/AudioPreflight.kt	Read-only audio and foreground/lock checks

Package / file	Responsibility
androidApp/voice/PhoneVoiceAdapter.kt	Recognition, TTS and conversation delivery
androidApp/resources/ResourceArbiter.kt	Microphone, playback and speech ownership
androidApp/data/SettingsRepository.kt	Atomic versioned storage and migrations
androidApp/noise/PhoneNoiseSampler.kt	Native microphone index without audio retention
androidApp/sleep/ScheduleAdapter.kt	Platform event delivery, cancellation and rescheduling
androidApp/wear/WearGateway.kt	Authenticated paired-device messages and deduplication
androidApp/ui/*Screen.kt	Layouts and ViewModels defined below
wearApp/voice/WatchVoiceAdapter.kt	Watch recognition and replies
wearApp/ui/*Screen.kt	Launcher, conversation, status and Stop

2. Data contracts

The following Kotlin declarations are interface sketches. Use platform-neutral serialization with explicit schema versions. Validate on import, load and command submission; never trust UI validation alone.

```
@JvmInline value class SlotId(val value: Int) // require 1..6
@JvmInline value class RequestId(val value: String) // UUID

enum class Capability { SUPPORTED, UNSUPPORTED, UNKNOWN }
enum class Control { VOLUME, SHARPNESS, TINNITUS }
enum class Origin { PHONE_TOUCH, PHONE_VOICE, WATCH_VOICE, NOISE, SLEEP }
enum class Provenance { TRANSMITTED, USER_CONFIRMED, USER_SUPPLIED }

data class ProgramConfig(
    val slot: SlotId,
    val name: String,
    val aliases: Set<String>,
    val enabled: Boolean,
    val tinnitus: Capability
)
data class Fitting(
    val id: String,
    val revision: Long,
    val profile: AcousticProfile,
    val leftAddress: Int,
    val rightAddress: Int,
    val controlAddress: Int,
    val programs: List<ProgramConfig>
)
data class KnownValue<T>(
    val value: T,
    val provenance: Provenance,
    val atEpochMillis: Long,
    val requestId: RequestId?
)
data class ProgramValues(
    val volume: KnownValue<Int>?,
    val sharpness: KnownValue<Int>?,
    val tinnitus: KnownValue<Int>?
)
data class RequestedState(
    val fittingRevision: Long,
    val program: KnownValue<SlotId>?,
    val values: Map<SlotId, ProgramValues>,
    val uncertainFields: Set<String>
)
```

Null means unknown, not zero. Unsupported tinnitus is represented by program capability, not a numeric sentinel. Unknown/uncertain state cannot supply a relative-command baseline. Values for an inactive program are historical; selecting it does not establish that its old values are current.

Store configuration revision separately from label-only updates where possible: changing addresses, waveform profile, slot identity or capability invalidates dependent assumptions. Renaming a display name updates references without pretending the slot changed identity.

```
sealed interface AidCommand {
    data class SelectProgram(val slot: SlotId) : AidCommand
    data class SetLevel(val control: Control, val value: Int) : AidCommand
    data class AdjustLevel(val control: Control, val delta: Int) : AidCommand
    data class Battery(val sides: Set<BatterySide>) : AidCommand
    data object Restore : AidCommand
}
data class CommandEnvelope(
    val id: RequestId,
    val origin: Origin,
    val command: AidCommand,
    val createdAtEpochMillis: Long,
    val expiresAtEpochMillis: Long,
    val expectedFittingRevision: Long
)
enum class Outcome {
    TRANSMITTED, ALREADY_REQUESTED, BUSY, BLOCKED,
    CANCELLED, FAILED, PARTIAL, UNKNOWN
}
data class StageResult(
    val name: String,
    val outcome: Outcome,
    val emittedFrames: Long,
    val completedAtEpochMillis: Long?
)
data class OperationResult(
    val id: RequestId,
    val outcome: Outcome,
    val stages: List<StageResult>,
    val reason: String?,
    val recovery: String?,
    val persistenceError: String?
)
```

Stable reason codes include INVALID_TARGET, UNKNOWN_PROGRAM, UNKNOWN_BASELINE, TINNITUS_UNAVAILABLE, TINNITUS_NOT_CONFIGURED, FITTING_CHANGED, LOCKED, APP_NOT_FOREGROUND, DND_ON, MEDIA_TOO_LOW, EXTERNAL_OUTPUT, AUDIO_MODE_BUSY, PHONE_UNREACHABLE, REQUEST_EXPIRED, PLAYBACK_INTERRUPTED and STORAGE_FAILED. Localize display/speech text outside domain code.

3. Phone screen layouts and event behavior

Use a single activity with navigation destinations Home, Voice, Programs, Saved settings, Noise, Sleep and Settings. Home has a persistent Stop control while any operation is active; all pages expose a microphone action. Back leaves a settings draft unsaved after offering discard/continue. Configuration Save is atomic. A failed write keeps the draft and reports failure.

Layouts below specify content order and grouping, not fixed pixel coordinates. Use accessible touch targets, scalable text, explicit labels and TalkBack traversal in visual order. Avoid color-only status. Compact screens scroll; Stop and microphone remain reachable. Landscape uses the same logical groups rather than changing command semantics.

3.1 HomeScreen

```
HeySig                    [Voice] [Settings]
Phone ready / blocked: <reason>     [Details]
LAST SENT PROGRAM
Sleep · slot 2 · 10:14 AM
Aid state not verified
[Universal] [Sleep] [Noisy]
[Outdoor]   [TV]    [Very Noisy]
Volume          [−]  8   [+]   [Set]
Sound Sharpness [−]  0   [+]   [Set]
Tinnitus        [−]  0   [+]   [Set]
[Left battery] [Both batteries] [Right battery]
[Saved settings] [Noise: Off] [Sleep: Off]
Operation: idle / stage and progress
[Microphone]                          [STOP]
```

Program buttons come from enabled fitting slots; no hardcoded names. Selected styling means last sent, not aid-confirmed. Unknown program displays “Program unknown — select a program.” Values show “unknown” or “historical” where applicable. Tinnitus displays the capability reason when unavailable. Program selection sends only a program command; it never reapplies stored levels.

Plus/minus invokes AdjustLevel with delta ±1. Numeric Set opens an integer editor with range and explicit Send. A draggable slider, if included, updates a local draft only; releasing it does not transmit. Send submits one absolute target. Display the old transmitted value until completion and show the pending target separately. Invalid targets disable Send and explain why.

Repeated settings return ALREADY_REQUESTED and expose Send again. Send again explicitly bypasses local deduplication for that user action and receives a new request ID. Battery requests always execute unless busy/blocked. Stop invokes coordinator.cancel and cancels the current conversation where relevant; it never sends a compensating aid command.

ViewModel events: SelectProgram(slot), DraftLevel(control,text), SendLevel(control), Adjust(control,delta), Battery(sides), SendAgain(previousTarget), OpenVoice, Stop, Open(destination). State: fitting, requestedState, audioReadiness, operation, automationSummary and local editor drafts.

3.2 VoiceScreen

```
Voice command             Source: phone / watch
Ready / Listening / Understanding / Clarifying
Heard: “set sleep time to ten”
HeySig: “Ten AM or ten PM?”
[Suggested answers, also speakable]
[Listen / Retry] [Repeat] [Back] [Cancel]
Result: <plain-language final outcome>
```

Show transcript only for the current session unless diagnostics are explicitly enabled. Listening starts only on user action or an enabled accepted wake trigger. Unknown speech offers Retry/Cancel. Retry performs fresh recognition. Show and speak clarification choices; Cancel/timeout discards draft changes. The same conversation reducer serves phone and watch. Voice can open any page, read every field, edit every setting and activate each action defined in this spec.

3.3 ProgramsScreen and FittingScreen

Programs list shows slot number, editable name, enabled flag, aliases and tinnitus capability. Row opens ProgramEdit with Name, Aliases, Enabled and Tinnitus supported/unsupported/unknown; Save and Cancel. Duplicate/ambiguous aliases fail Save with the conflicting slots named.

Fitting screen groups Addresses (control/left/right), waveform profile, program mapping and validation status. Profile initially uses the tested values only after user review: 44100 Hz, base 15360 Hz, spacing 512 Hz, bitSeconds 0.0625, gainDivisor 1.2, address-XOR on. Tested address 12 is offered as this user's setup, never a generic default.

Advanced profile edits show a confirmation summary and invalidate fitting validation. Validate profile constraints through AcousticProfile before persistence. Addresses accept decimal or explicit 0x hexadecimal, range 0..255. A guided test requires selecting a configured program, explicit Send test, and user response Heard expected response / No response / Unsure. Record feedback without claiming electronic acknowledgment. No automatic address scans or max-level tests.

Voice sequence: “configure program two” → read fields → “name Sleep” → “tinnitus supported” → “review” → read draft → “save” → commit. “Set aid address to twelve” asks whether common/control or individual side; “use twelve for all” proposes all three and requires confirmation. Permission setup uses spoken Android instructions where system UI interaction is required.

3.4 SavedSettingsScreen

Show saved program, known/unknown levels, tinnitus capability, fitting revision, provenance and saved time. Actions: Save requested settings, Restore, Enter external settings, Export, Import. Save does not interrogate aids or Signia. Unknown fields remain unknown.

Restore opens preview listing stages to send and fields to skip. Incomplete record requires “Restore known values” confirmation. A fitting mismatch blocks before any stage. During restore show each stage Pending / Transmitting / Transmitted / Skipped / Failed. Stop leaves completed stages visible; retain the original snapshot. Do not label partial restoration success.

Enter external settings is a draft editor with program and optional levels; voice asks each field and accepts “unknown.” Saving marks supplied fields USER_SUPPLIED. Export/import initiates a system picker with spoken guidance; import parses and previews before replacing data. Cancel leaves all records unchanged.

3.5 NoiseScreen

```
Noise                    Source: phone microphone
[Off] [Observe] [Automatic]       [Stop]
Index: −46.2   Sample age: 4 s   [Sample now]
Classified: Noisy → program 3
Candidate: Noisy 2/3 · Cooldown: 14 s
Blocked by: override / sleep / microphone / none
[Calibrate] [Settings] [Read status]
```

Settings fields: Quiet role, optional Noisy role, Very noisy role, low/high thresholds, hysteresis, confirmations (1..20), cooldownSeconds, captureSeconds, waitSeconds and freshnessLimitSeconds. Initial timing choices 60/5/10 seconds; thresholds require new native-sampler measurement. Require finite numeric values, low < high, margin > 0, 2×margin < high−low, confirmations 1..20 and positive timing values. Initial supported timing bounds: cooldown 0..3600, capture 1..10, wait 1..300, freshness 2..900 seconds; freshness must exceed capture + wait. These are proposed app bounds, not inherited protocol limits.

Save stops sampling, clears candidates and leaves mode Off. Automatic remains unavailable until fitting roles and calibration direction are confirmed. Observe never sends commands. Stop prevents further decisions immediately, then releases any active capture. Sample now reports busy if capture resources are unavailable.

Calibrate wizard: explain index → capture quiet → capture typical → capture loud → read results → confirm larger index means louder → propose thresholds → review/save. Do not auto-enable switching.
Unsuitable/nonseparated measurements require another capture or explicit manual thresholds; never label them calibrated sound-pressure dB.

Voice supports all fields, role choices, raw sample, mode changes, candidate/cooldown queries, calibration steps and Stop. “Start automatic noise” reads current roles and asks confirmation if configuration has not yet been reviewed in this session.

3.6 SleepScreen

Fields: Enabled, Sleep program, Sleep local time, Wake program, Wake local time, runtime Sleep-active status, next scheduled event and last event outcome. Actions: Save choices, Enable, Disable, Sleep now, Wake now, Read schedule.

Programs are required enabled slots; times use LocalTime and must differ. Display 22:00/07:00 suggestions without saving until confirmed. Save disables schedules and clears sleep-active. Enable schedules future occurrences only. Sleep now/Wake now require enabled valid configuration and no override. Bare “sleep” invokes Sleep now; “program Sleep” invokes direct program selection.

Voice time parsing accepts 24-hour values and explicit AM/PM. “Ten” is ambiguous: ask AM/PM. “Tomorrow” is not a recurring daily time and must be clarified. Show timezone explicitly. Failures show missed/deferred status without replaying later.

3.7 SettingsScreen and DiagnosticsScreen

Voice settings: prompts on/off, results on/off, reply device (initiating device/phone/watch), optional wake enabled on each device. Automation controls: override on/off, Cancel automation and Resume reviewed modes. Data controls: export/import configuration, mark requested state unknown and delete diagnostics. Destructive deletion/reset requires a read-back confirmation; Back/Cancel preserves data.

Readiness page shows fitting validity, phone unlocked/foreground, DND, media level/required level, audio mode and detected output conflicts. Check audio reads only; it sends no tones and changes no Android settings. Diagnostics shows recent bounded events and last operation stages; Explain failure produces the same recovery message on screen and voice.

4. Watch layouts and delivery

4.1 Launcher and conversation

```
HeySig
Phone: reachable / unavailable
[Speak command]
[Read status] [Stop]
```

Tile provides Speak and Stop entry points. Main watch screen adds current last-sent program and automation summaries. Conversation screen shows Listening, recognized text, clarification prompt and response options with large Listen/Cancel actions. Long option lists are read in short groups with “next choices” and “repeat”; no phone-only settings step is allowed except required Android system interaction.

Watch captures recognition, then forwards the recognized text and conversation ID. Phone parses and executes; watch speaks final replies locally by default. Phone speech is selected only by explicit reply preference. If watch TTS is unavailable show text/haptic status and offer spoken phone output when feasible; do not silently lose the result.

4.2 Message contract

```
data class VoiceRequest(
    val requestId: String,
    val conversationId: String,
    val text: String,
    val createdAt: Long,
    val expiresAt: Long,
    val expectedConfigRevision: Long
)
data class VoiceReply(
    val requestId: String,
    val sequence: Long,
    val phase: String, // RECEIVED, CLARIFY, RUNNING, FINAL
    val displayText: String,
    val spokenText: String?,
    val result: OperationResult?
)
```

Only paired/authorized watch nodes may submit requests. Use a 30-second delivery expiry as an initial design value, checked against the receiving phone clock with clock-offset handling at connection setup; if time agreement is unavailable, require a fresh live session before control. Persist command IDs/outcomes for deduplication across restart. Retain at most 256 records for 24 hours; expired unseen requests never execute. IDs whose execution was interrupted return UNKNOWN and require an explicit new user request.

Send the same request ID when retrying delivery, not a new control request. Repeated ID returns stored progress/result without replay. Sequence numbers suppress old replies. Watch may show “Phone has not replied” after five seconds but must not infer failed playback or automatically resend as a new command. Reconnection retrieves result by ID; it does not enqueue old speech for later execution. Settings conversations also reject stale revisions.

Stop has its own ID and identifies the target operation/session. It bypasses normal busy rejection. A disconnected watch says Stop could not reach the phone; it must not claim cancellation.

5. Function contracts and execution

5.1 Public application functions

Function	Inputs	Return and required behavior
parseCommand(text,context)	Text, active conversation, fitting aliases	Parsed(intent), Clarify(options), Cancel or Unsupported; pure, no side effects
reduceConversation(state,event)	Current draft and response	New state plus effects; never persist before explicit Save/Confirm
submit(envelope)	Validated ID/origin/target/revision	Operation handle or Busy/Blocked; starts at most one operation

Function	Inputs	Return and required behavior
cancel(operationId)	Target ID	Cancellation acknowledgment plus eventual final result; stop pending stages
getStatus()	None	Requested state, runtime modes, readiness, ages and provenance; no playback
setProgram(slot,force)	Enabled fitted slot, resend flag	Map slot → PCM; update program only after full transmission
setLevel(control,value,force)	Known program and valid level	Capability/range checks, one absolute command, update that field only
adjustLevel(control,delta)	Delta ±1	Resolve from valid same-context baseline; reject unknown/out-of-range
checkBattery(sides)	Nonempty left/right set	Left then right; stage results, no numeric battery state
saveSnapshot()	Current requested state	Atomic record or storage error; preserve unknown fields
previewRestore()	Existing snapshot and fitting	Validated stages, skipped fields, mismatch errors; no audio
restore(planToken)	Confirmed plan bound to revision	Serialized stages; revalidate before each stage; preserve snapshot
saveFitting(draft,revision)	Valid draft + expected revision	Atomic update, dependent invalidation and automation stop
configureNoise(draft)	Valid settings	Atomic update, mode Off and candidates reset
startNoise(mode)	Observe/Automatic	Validate configuration/resources; start exactly one loop
stopNoise()	None	Stop future decisions immediately, release capture, clear candidate
setOverride(enabled)	Boolean	Persist preference, clear pending noise; enabling blocks sleep/noise
configureSleep(draft)	Programs and local times	Atomic update, disable schedule, clear sleep-active
enableSleep(enabled)	Boolean	Validate and arm/disarm future events; no immediate selection
requestSleepState(state)	Asleep/Awake	Guard enabled/override; select configured program then update runtime state
checkAudio()	Current platform state	Ready or ordered blocking reasons; read-only
markUnknown(fields)	Explicit affected scope	Clear current assumptions, keep historical log and saved record

All public functions return domain results rather than throwing raw Android errors to the UI. Internal exceptions become typed failures while preserving cause details in bounded diagnostics. A cancellation exception must still run resource cleanup and finalize uncertain playback state.

5.2 Coordinator algorithm

1. Validate expiry, expected fitting revision and target. Reject invalid/busy work before allocating playback resources. Queries and Stop remain available.
2. Resolve a typed plan. Normalize relative changes into absolute targets before playback; bind the plan to program/fitting context.
3. Reject local repeated targets as ALREADY_REQUESTED unless force was explicitly requested. No repeat suppression for battery.
4. Acquire the shared resource arbiter, pause existing capture/listening, and recheck audio/fitting/program context. A confirmation dialog never owns playback resources.
5. Persist an operation-start journal entry before emitting audio. Failure to establish the journal blocks transmission, avoiding untracked duplicate work after restart.
6. Render the stage through the validated PCM adapter; stream through built-in speaker. Track submitted and played frames. A successful write alone is insufficient: wait for the full frame count and normal completion.
7. Emit stage completion and update only its requested-state field. Persist requested state; report a write failure separately if playback already completed. Never replay to fix storage.
8. Between stages observe cancellation and recheck guards/revision. For both batteries use a validated separation; initially configure one second after completion, pending physical acceptance. Restore uses the same configurable stage gap, not an assumed aid acknowledgment.
9. Finalize operation journal/result, release AudioTrack/focus/resources in finally, and resume only modes that were previously enabled and remain valid. Clear automation candidate history after interrupted resource ownership.

Default stage timeout is max(5 seconds, PCM duration + 3 seconds); no unbounded playback wait. A later stage never begins after timeout. Mark partially emitted state uncertain. On startup, unfinished journal entries become UNKNOWN and active modes become Off; no playback resume.

5.3 Acoustic adapter contract

```
interface AidTransport {
    suspend fun transmit(
        stage: AcousticStage,
        onProgress: (playedFrames: Long) -> Unit
    ): PlaybackResult
    suspend fun stop()
}
interface AudioPreflight {
    fun inspect(): AudioReadiness
}
```

Reuse the existing ProtoRequest encoder unchanged behind the adapter until PCM regression gates pass. Commands: program = 0x40 + slot − 1; volume = 0x1f − value; sharpness = 0x20 + (value AND 15); tinnitus = 0x3f − value; left battery 0x80; right 0x81. These mappings apply to the tested fitting/platform, not every Signia model.

Use controlAddress for program/levels, leftAddress/rightAddress for battery. Validate address and profile independently. Stream signed little-endian mono PCM16 at the configured sample rate through the proven media playback path. Do not change audio gain/profile parameters based on environmental noise. Phone volume, DND and incompatible outputs produce instructions, not automatic changes.

The initial release retains the prototype's unlocked/foreground constraint until separate locked/background tests qualify a revised policy. Watch commands and schedule events report this block honestly. Signia itself need not be open.

6. State transitions and automation functions

6.1 Command and conversation states

Current state	Event	Next state / action
Idle	Valid request	Preflight → Rendering → Playing
Playing	Full completion	Persisting → Transmitted
Playing	Stop/timeout/output loss	Stopping → Cancelled/Failed; affected field uncertain
Any active operation	Another control	Return Busy without replacing active operation
Awaiting clarification	Response	Update draft or dispatch resolved intent
Awaiting confirmation	Confirm	Revalidate revision then commit/submit
Any conversation	Cancel/timeout	End; discard unsaved draft; no aid command
Startup	Unfinished operation journal	Mark Unknown; do not replay

Manual program transmission enables override; failed/blocked selection does not. A manual program command takes priority over pending noise/sleep work, but does not interrupt already emitting audio automatically. Stop is explicit. Override blocks both sleep and noise; releasing it does not enable either mode. Sleep-active suppresses noise. Manual level commands do not implicitly enable program override; this matches the specific manual-program priority rule.

6.2 NoiseReducer

```
fun reduceNoise(
    state: NoiseState,
    event: NoiseEvent,
    settings: NoiseSettings,
    nowMonotonicMillis: Long
): NoiseDecision
// decision = next state + None | Propose(slot) | RequestProgram(slot)
```

NoiseState contains mode, classified band, candidate slot/count, last fresh sample time, last transmission time and blocking reason. NoiseEvent includes FreshSample(index), CaptureFailed, SamplingGap, GuardBlocked, ManualOverrideChanged, SleepChanged, CommandCompleted and Stop. Reducer performs no Android work.

Convert the sampler index once into “larger means louder.” Initial band: below L quiet, L through H middle, above H loud. From quiet enter middle above L+M and loud above H+M; from middle enter quiet below L−M or loud above H+M; from loud enter middle below H−M and quiet below L−M. Equality stays in the current band. Check outer-band jumps first. If middle role is absent, middle samples clear candidates.

Count consecutive fresh samples for the same target; a target change restarts at one. Once confirmations and cooldown qualify, Observe emits Propose; Automatic emits RequestProgram. When the target is already last requested, emit no transmission and do not extend cooldown. A failed/blocked request, stale gap, Stop, override, sleep or microphone interruption clears pending count. Cooldown uses monotonic time and starts after completed transmission only. After cooldown expiry require a new confirmation sequence; do not retain pre-cooldown strikes.

Sampler uses one bounded capture at a time; wait cadence starts after capture completion. Exclude any capture overlapping speech or command tones. Native index calculation, microphone source and hardware gain behavior must be documented and calibrated before shipping Automatic. A concrete initial implementation is RMS over PCM samples, converted to 20×log10(max(rms,epsilon)) relative to full scale; epsilon prevents log(0). Do not reuse AutoVoice thresholds or label the value SPL. Invalid/empty captures are failures, not silence samples.

6.3 SleepPlanner

```
fun nextOccurrences(
    settings: SleepSettings,
    now: Instant,
    zone: TimeZone
): List<ScheduledOccurrence>
suspend fun handleOccurrence(id: String): OperationResult
```

Occurrence identity includes fitting/schedule revision, local date and event type. Use the current timezone to plan the next sleep and wake. In a DST repeated time choose the first occurrence only; in a nonexistent local time mark that day's event skipped. Recompute on timezone/clock changes, cancelling old platform registrations first. Keep processed occurrence IDs so a broadcast cannot execute twice.

At delivery reload persisted configuration, verify enabled/revision/time validity and that the occurrence is due and not already processed. Initial lateness tolerance: 60 seconds; beyond it record missed and do not transmit. Within it all normal guards still apply. Blocked/busy events are missed, not queued. A successful sleep command sets SleepActive; successful wake clears it. No state update for failed or partial playback. Disable/Save explicitly clears suppression.

Platform scheduling APIs and permissions are implementation feasibility work. If the OS cannot provide the required timing, expose “schedule unavailable” and recovery guidance; do not show Enabled after registration fails. On restart retain saved choices and last event history but leave runtime scheduling off for explicit resume.

7. Voice intents and guided function coverage

Every screen event maps to a typed AppIntent. Control intents use AidCommand; settings intents update conversation drafts. Maintain one registry containing canonical phrases, aliases, parameter types, required context, confirmation policy and reply templates. Generate help from this registry so phone/watch behavior cannot drift.

Intent family	Required phrases and dialog behavior
Navigation/help	“open noise settings,” “list programs,” “help with sleep,” “repeat,” “back,” “cancel”

Intent family	Required phrases and dialog behavior
Programs	“program two,” “set program to Sleep,” “current program,” “send again”
Levels	“volume ten,” “balance minus three,” “tin four,” mute/max/soft/sharp, current queries, up/down
Batteries	“battery check,” “check left battery,” “check right battery”
Snapshots	“save settings,” “what is saved,” “restore settings,” “enter external settings”
Fitting	“configure aids,” “set control address twelve,” “rename program two,” “set tinnitus availability”
Noise	“noise sample,” “observe noise,” “automatic noise,” “stop noise,” “calibrate noise,” edit/read every field
Sleep	“configure sleep,” “sleep at ten PM,” “wake at seven AM,” enable/disable, sleep/wake now
Priority	“override on/off/toggle,” “manual/automatic mode,” “cancel automation,” “resume automation”
Voice	“prompts off,” “spoken results on,” “reply on watch,” “start/stop Hey Sig listening”
Recovery/data	“check phone audio,” “explain last failure,” “mark settings unknown,” import/export/delete diagnostics

Normalize whitespace/case, preserve configured program punctuation, accept one trailing sentence punctuation character, and resolve numeric words including minus/negative. Match exact configured names/aliases before broad keywords. “Program Sleep” always means program selection; bare “sleep” means sleep-state action. Do not guess unrecognized aid-control targets. Ambiguous input becomes Clarify without playback. Optional local-model interpretation follows the proposal, validation and confirmation gates in section 10.

Example full watch conversation: “Configure noise” → “Choose quiet program” → “Universal” → “Choose noisy program” → “Noisy” → “Choose very noisy program” → “Very Noisy” → review thresholds/calibration → “Save” → “Saved. Noise automation is off.” Follow the same reducer on phone. Editing only one field should not force reentry of every other valid field.

For setup/security/system limitations, say exactly which device needs interaction: “Unlock your phone and open HeySig, then say try again.” Watch voice access remains available to status/settings even if playback is blocked. Never translate a blocked action into silent success.

8. Storage, permissions and failure recovery

Use versioned records for fitting, voice preferences, noise settings/calibration, sleep settings, requested state, snapshot and operation journal. Repository mutations require expected revision to prevent stale watch/phone drafts overwriting newer edits. On conflict return the current revision and ask the user to review again.

Write a temporary record in the same private directory, flush and atomically replace. Keep the last valid backup where supported. A malformed/unsupported schema loads no guessed defaults into active automation; report recovery, retain the original for export and leave affected functions disabled. Settings saves and automation-state changes must be serialized with dependent invalidations.

Microphone, speech service availability, notification/background operation, watch connection and scheduling permissions have separate readiness states. Explain why each is needed at the feature entry point. Denial must not block unrelated manual controls. Permission revocation stops affected resources and marks modes stopped. Never claim granting permissions can be completed by an app voice command alone.

Keep logs bounded (initial 256 KiB rolling numeric/event log plus last 256 operation records). Store no microphone audio or persistent recognition transcript by default. Export strips secrets/device identifiers unless explicitly included. Diagnostic deletion affects diagnostics only, not fitting or restoration records.

9. Implementation sequence and test specification

Build in vertical phone/watch slices rather than completing phone functions and deferring watch parity:

1. Shared models, storage, fitting review, Home screen, coordinator and validated PCM adapter. Phone/watch push-to-talk can select programs and request battery.
2. Level controls, status provenance, snapshots and their full voice dialogs. Add screens and explicit restore-stage reporting.
3. Settings, fitting edits, voice preference changes and recovery/export dialogs on both devices. Complete watch deduplication/reconnect tests.
4. Native noise sampling, calibration, observation, then Automatic and priority tests.
5. Sleep scheduling, lifecycle recovery, locked/background feasibility and combined resource tests.
6. Optional hands-free wake, endurance and release packaging. Wake detection implementation/license remains a decision requiring validation, not an assumed OS capability.

Test ID	Action	Required result
C01	Phone and watch say “program two”	One slot-2 transmission, last-sent Sleep; record user beep feedback separately
C02	Repeat C01, then “send again”	No-op first; exactly one new transmission on explicit resend
C03	Set volume with unknown program	Ask program; no PCM until resolved
C04	Set known volume then relative up	Correct absolute mapping; out-of-range rejected
C05	Unsupported/unknown tinnitus	No command; unavailable/configure capability response
C06	Both batteries, Stop after left	No right playback; result reports left stage and cancellation
C07	DND/low media/external route	Block with device-specific instruction; no audio configuration mutation
C08	Kill process during transmission	Restart shows uncertain field; no replay
S01	Restore incomplete snapshot	Preview skipped fields, explicit confirmation, exact serialized known stages
S02	Storage failure after playback	Report transmitted plus save failure; preserve previous record
V01	Every screen field/action by both voice paths	Same typed effect and understandable local reply
V02	Ambiguous time/name, Cancel, silence	Clarification/discard; no unintended write or playback
W01	Duplicate request, late reply, reconnect	One execution; ordered result recovery; no stale queued command
W02	Watch disconnected during Stop	State cancellation not confirmed; retrieve phone result later

Test ID	Action	Required result
N01	Boundary oscillation and outer-band jump	Exact hysteresis/confirmation behavior specified in section 6
N02	Voice/tones overlap sampling	Capture excluded, candidate reset, previous mode resumes safely
N03	Override/sleep/gap/cooldown	No premature switch; require fresh sequence
T01	DST/timezone/duplicate delivery	One valid occurrence or explicit skipped status; never double play
T02	Restart or delayed blocked event	No automatic replay; choices retained, modes visibly Off/missed
R01	Eight-hour observe and automatic trials	Detect crashes/restarts/gaps; bounded memory; final stop recorded

Unit tests target parser ambiguity, reducers, validation, snapshots, scheduling and priority. Audio regression tests compare generated PCM to frozen evidence. Device acceptance tests cover actual recognition, watch transport, audio routes, user aid response, interrupted stages and no-ADB operation. Mocked success never substitutes for these physical checks.

Release checklist: every described function has implemented screen and phone/watch voice coverage; no current-state claims without provenance; no Tasker/plugin runtime dependency; permissions and recovery work; source builds reproducibly; limitations match measured locked/background/wake capability. iPhone and watch-to-aid playback are outside this implementation.

10. Phased voice improvement

Implement these phases on `main`. V1–V3 strengthen the mandatory phone/watch voice implementation in section 9; V4–V5 are optional enhancements and must not delay or replace core voice parity. Recognition converts sound to text; interpretation converts text to an intent. Correction learning improves local interpretation and vocabulary, not the weights of Android's speech model.

V1. Establish recognition baseline and vocabulary support

Add `RecognitionAdapter`, `RecognitionCapabilities`, `RecognitionHypothesis` and `VoiceEvaluationRunner`. Each adapter reports on-device availability, offline-language/model readiness, phrase-hint support, alternatives and confidence availability. Missing confidence is unknown, not zero or certainty. Probe actual phone/watch services rather than assuming identical support. Offer explicit download/setup guidance when supported; never silently change offline preference to cloud recognition.

Build phrase hints from enabled program names, confirmed aliases and HeySig control vocabulary. Refresh after fitting/alias changes. Evaluate multiple returned hypotheses through the deterministic parser. If valid alternatives disagree on program, number, sign or action, ask clarification; never select the most permissive interpretation. Keep recognition and command execution separately measurable.

Deliverables: adapter contracts, vocabulary builder, capability/status UI, phone/watch baseline corpus and numeric aggregate evaluation. Voice settings adds Recognition mode (on-device preferred / on-device required / disclosed system service), Recognition readiness and Test recognition. Voice can read/change these fields through both devices.

Exit gate: every core command family has live phone and watch samples, including program names, tinnitus, negative sharpness, numbers and cancellation. Record device/service/version, quiet/noisy conditions, expected intent, first-attempt intent accuracy, clarification/correction rate and latency. Report each device separately. Tests

require no aid playback merely to score recognition.

V2. Learn explicitly confirmed personal corrections

Add `CorrectionRepository` and `CorrectionService`. A voice correction such as “No, I said Sleep” enters clarification, displays/speaks the proposed mapping, and asks “Remember this correction?” A generic Yes to an execution confirmation is not consent to store a correction. Correcting interpretation does not undo earlier playback: report what already transmitted and ask before issuing a new command. Never automatically replay after a correction.

A correction record contains ID, normalized mistaken phrase, canonical intent template or alias, applicability context (device source, fitting ID/revision and conversation field), explicit confirmation time, enabled flag and schema version. Parameterized corrections cannot invent numeric values or signs. Prefer narrow exact phrase/context matching; do not learn a broad substring rule from one example. If an existing exact valid command conflicts, require explicit alias review or reject the correction. Ambiguous corrections remain inactive.

Store confirmed mapping text locally only after opt-in; raw audio is never required. This is a narrow exception to default non-retention of recognition transcripts: retain only the confirmed phrase mapping, not conversation history. Cap at 200 mappings initially; do not silently evict user-confirmed mappings. Ask the user to review when full. Export/import corrections is separately selectable and requires preview.

Add Voice settings → Personal corrections: Enabled, List/read mappings, Edit, Forget one, Clear all and Export/import. Voice phrases include “remember that correction,” “list my corrections,” “forget that correction,” and “reset learned phrases.” Clear all requires confirmation; Cancel preserves mappings. Fitting identity changes invalidate dependent corrections until reviewed. Phone/watch share the phone-owned repository, retaining source context where useful.

Exit gate: explicit consent, restart persistence, device/context scoping, conflict rejection, revision invalidation and voice deletion pass on phone/watch. Forgotten mappings stop affecting interpretation immediately. Replayed test phrases show improvement without changing accepted numeric/program commands incorrectly.

V3. Continuous evaluation and controlled improvement

Add a versioned `VoiceEvaluationCase` corpus and `VoiceMetricsRepository`. Record aggregate counts by source, command family and resolver version: attempts, correct first intents, unsupported intents, clarification, explicit correction, cancellation and latency. A lack of user complaint does not label an interpretation correct. Ground truth comes from explicit correction/confirmation or a deliberately scored test. Distinguish user satisfaction from actual aid reception.

Default operation stores numeric aggregates and resolver versions without transcripts/audio. An opt-in “add this example to voice tests” workflow previews the exact text, expected intent and context before saving a local example. Allow voice review, deletion and explicit sanitized export. Keep a held-out set apart from examples used to add aliases/corrections; score proposed updates against both sets.

Promote parser/alias/model changes only after comparing the same baseline and held-out cases. Zero unintended dispatches on the ambiguity, numeric/sign and conflicting-target test set is mandatory; improvement in one family must not conceal a regression in another or on the watch. Report first-attempt accuracy, false dispatch count, clarification rate and p50/p95 latency with sample counts. Establish numerical performance budgets from V1 measurements before selecting an LLM. Version changes and allow rollback/reset. Do not automatically rewrite parser rules or train a model from unreviewed live speech.

Exit gate: a reproducible before/after report for phone/watch, a tested rollback, aggregate deletion/reset and regression checks. Continuous improvement means a repeatable review-and-measure process; it does not promise that recognition gets better after every utterance.

V4. Optional local LLM feasibility experiment

Run a small language model on the phone for unfamiliar wording, clarification drafts and explanation of structured errors. The watch forwards its recognized text through the same phone-side interpreter. Keep deterministic exact commands and Stop on the fast path. Local LLM support is off by default until qualified; core commands work without a model, network or completed model download.

Evaluate a maintained Android-capable runtime such as LiteRT-LM; select the exact runtime, model, quantization and license after testing on the user's Pixel. Model installation is an explicit download with size, storage and offline behavior disclosed. Verify model integrity, loading failures, available memory, cold/warm latency, battery use, thermal behavior and microphone/audio contention. Do not assume the Pixel's built-in assistant model is available to this app.

Deliverables: `IntentInterpreter`, `LocalLlmInterpreter`, `ModelManager`, benchmark report and a decision to accept, defer or reject. Runtime and model versions must be pinned for reproducibility. Compare with the deterministic resolver on held-out natural-language cases. No model-selected command executes during initial evaluation: show its proposal only. Inspect proposals such as “one step softer” for required volume-versus-sharpness clarification.

Exit gate: measurable natural-language benefit over V3, acceptable measured device resource use and latency budgets, no regression in exact commands, clean timeout/cancellation/unload, and full offline testing after installation. If gates fail, keep V1–V3 and document the result.

V5. Validated optional interpretation integration

Pipeline: recognition hypotheses → exact parser and confirmed contextual corrections → optional LLM proposal → schema/context validation → clarification/confirmation when required → existing coordinator. The LLM has no AudioTrack, repository-write, permission or aid-command execution access. It cannot change fitting capability, invent a baseline, reinterpret actual last-sent state as verified, or override preflight.

`IntentInterpreter.propose(input,context)` returns `Proposal(intent,evidence)`, `Clarify(options)`, `Unsupported` or `Unavailable`. Context contains configured names/aliases, relevant conversation draft and structured status, not arbitrary raw logs. Require a bounded schema with allowlisted intent types and parameters. Validate ranges, program/capability, revision and baseline in normal code after interpretation. Reject malformed/unknown fields and missing parameters. Model-generated confidence is not evidence of correctness. Cap inference at five seconds initially; timeout/unavailable returns deterministic help or clarification and never a guessed command.

An LLM-only aid-changing proposal must first be read back and explicitly confirmed. Ambiguous numeric values, signs, program names or control families always require clarification. Settings proposals update drafts and still require Save. Help/error explanations may be conversational but must be grounded in structured result facts; never manufacture successful reception or battery percentage.

Voice settings adds Local language assistance on/off, Model status, Download/remove model and Read performance status. Turning it off stops pending inference and preserves ordinary voice operation. Model removal requires confirmation. Phone and watch must report missing model/disconnection/timeout clearly. Stop cancels inference immediately; delayed output from a cancelled or expired session is discarded.

Exit gate: full phone/watch end-to-end tests, schema/fitting/range and ambiguous-number rejection, no duplicate dispatch on retries, cancellation under load, no inference during acoustic playback and no command-state changes when the model fails. Document accepted hardware and exact model/runtime versions. Local LLM interpretation does not imply speech-model retraining or hands-free wake capability.

10.1 Additional acceptance cases

Test ID	Scenario	Expected result
VI01	Program renamed; hint service unsupported	Vocabulary refreshes; deterministic resolver remains usable; no false support claim
VI02	Hypotheses disagree: volume four / fourteen	Clarification before dispatch
VI03	“No, I said Sleep” after TV transmitted	Report prior transmission; confirm new action and separate learning consent
VI04	Learn/forget a watch-context correction	Persist only confirmed scope; forgetting immediately restores baseline behavior
VI05	Fitting changes or correction conflicts	Invalidate/review mapping; no guessed target
VI06	Compare candidate against held-out corpus	Report phone/watch metrics and reject regressions
VI07	Model missing, timeout, memory failure	Existing voice path remains usable; no aid command from failed inference
VI08	LLM proposes invalid number or unsupported tinnitus	Normal validation rejects; model cannot bypass guards
VI09	Cancel while inference running, then late answer	Discard answer; no execution or saved draft
VI10	Disable assistance and delete examples	No model calls; chosen local data removed; ordinary settings retained

Technical references for implementation evaluation: Android SpeechRecognizer, RecognizerIntent options and Google LiteRT-LM. Service/device support and runtime APIs must be rechecked during implementation.